HumAL ,Äî Active Learning Platform API Reference
RAG Knowledge Document for HumAIne Swarm Chatbot

Source: HumAL GitHub repository (https://github.com/humaine-jsi/humal)
Purpose: Enables the HumAIne Swarm chatbot to answer questions about HumAL's Active Learning architecture, setup, and workflow ,Äî including how it relates to the hackathon notebook tasks.
HumAIne project: Grant Agreement 101120218

---

1 ,Äî What is HumAL

HumAL (HumAIne Active Learning Platform) is the integrated Human-in-the-Loop machine learning system developed within the HumAIne project's smart ticketing pilot. It provides a complete pipeline for:

- **Automated ticket classification** ,Äî routing support tickets to the correct team using a trained ML classifier
- **Active Learning** ,Äî interactive model training where the system queries which tickets a human operator should label next, maximising learning efficiency with minimal labeling effort
- **Resolution Generation** ,Äî LLM-powered ticket resolution suggestions using Retrieval-Augmented Generation (RAG) over a knowledge base of past resolved tickets
- **Explainable AI (LIME)** ,Äî LIME-based explanations of individual routing decisions, showing which words in a ticket drove the classification
- **Modern web UI** ,Äî React + TypeScript frontend with real-time updates

HumAL directly implements the HumAIne HITL (Human-in-the-Loop) paradigm: the AI system identifies which unlabeled tickets are most informative, and a human operator provides labels, closing the feedback loop.

---

2 ,Äî Architecture

2.1 Technology Stack

Layer	Technology
Frontend	React + TypeScript + Vite (port 5173)
Backend	FastAPI + Python 3.8+ (port 8000)
ML / AL	scikit-learn, modAL-python
XAI	LIME (`lime` library)
LLM / RAG	OpenAI API
Package management	`uv` (recommended) or `pip`
GPU support	CUDA (auto-detected, optional)
Deployment	Docker Compose

2.2 Project Structure

```
HumAL/
,îú,îÄ,îÄ backend/
,îÇ   ,îú,îÄ,îÄ app/              # FastAPI application
,îÇ   ,îú,îÄ,îÄ data/             # CSV datasets (place files here)
,îÇ   ,îú,îÄ,îÄ models/           # Saved ML models (place folders here)
,îÇ   ,îî,îÄ,îÄ tests/            # Backend tests
,îú,îÄ,îÄ frontend/            # React + Vite frontend
,îú,îÄ,îÄ start-dev.bat        # Windows startup
,îú,îÄ,îÄ start-dev.sh         # Unix/Linux startup
,îú,îÄ,îÄ requirements.txt     # Python dependencies
,îî,îÄ,îÄ install.py           # Automated installer (handles CUDA
detection)
```

2.3 Access Points (after startup)

Service	URL
Frontend Application	http://localhost:5173
Backend API	http://localhost:8000
API Documentation (Swagger)	http://localhost:8000/docs

---

3 ,Äî Available Pages and Their Functions

Page	Route	Description
Home	`/`	Landing page
Training	`/training`	Model training interface ,Äî trigger AL training cycles
Dispatch Labeling	`/dispatch-labeling`	Human labeling interface ,Äî the core HITL step
Ticket Resolution	`/ticket-resolution`	LLM-generated resolution suggestions using RAG
Inference	`/inference`	Run model inference on new tickets

3.1 The AL Workflow in HumAL

The Active Learning loop in HumAL follows this sequence:

1. **Dispatch Labeling page** (`/dispatch-labeling`): The system presents unlabeled tickets to the human operator. It surfaces the most informative tickets first (those the current model is most uncertain about ,Äî uncertainty sampling). The operator assigns a team label to each ticket.

2. **Training page** (`/training`): After labeling a batch of tickets, the operator triggers model retraining. The model is updated with the newly labeled examples.

3. **Inference page** (`/inference`): The updated model routes new incoming tickets automatically.

4. **Cycle repeats**: As more labeled data accumulates, model accuracy improves iteratively.

This matches the 4-step AL cycle in the hackathon notebook: Query ,Üí Retrieve Labels ,Üí Teach ,Üí Remove from Pool.

---

4 ,Äî Data Requirements

4.1 Required Files (place in `backend/data/`)

File	Purpose
`User_Request_last_team_ANON.csv`	Resolution knowledge base for RAG
`al_demo_train_data.csv`	Training ticket texts for the AL demo
`al_demo_test_data.csv`	Test ticket texts for evaluation
`al_demo_train_labels_dispatch.csv`	Training labels for dispatch (team assignment)

4.2 Required Models (place in `backend/models/`)

Folder	Purpose
`perfect_team_classifier/`	Pre-trained high-accuracy team classifier
`ticket_classifier_model/`	Iteratively trained AL classifier

---

5 ,Äî Setup and Installation

5.1 Local Development (Python + Node)

```bash
# 1. Create and activate virtual environment
uv venv al_api_venv           # recommended
# or: python -m venv al_api_venv
al_api_venv\Scripts\activate   # Windows
source al_api_venv/bin/activate # Unix/Mac

# 2. Install all dependencies (auto-detects CUDA)
python install.py
# CPU only: python install.py --cpu-only
# Force pip: python install.py --use-pip

# 3. Place data and models in backend/data/ and backend/models/

# 4. Configure environment
copy .env.example .env   # Windows
cp .env.example .env     # Unix/Mac
# Edit .env and add: OPENAI_API_KEY=your-key-here
```

```
# 5. Start the application
.\start-dev.bat    # Windows
./start-dev.sh     # Unix/Mac
```

5.2 Docker Deployment

```bash
# 1. Configure environment
cp .env.example .env
# Edit .env: OPENAI_API_KEY=your-key-here

# 2. Start all services
docker-compose up

# Services available at:
#   Frontend: http://localhost:5173
#   Backend:  http://localhost:8000
#   API docs: http://localhost:8000/docs
```

5.3 Environment Variables

Variable	Required	Default	Description
`OPENAI_API_KEY`	Yes	,Äî	OpenAI API key for LLM resolution generation
,Äî	,Äî	,Äî	Other variables configured in .env.example

---

6 ,Äî Relationship to the HumAIne Hackathon Notebook

The hackathon notebook (`humaine_al_hackathon.ipynb`) implements the same AL workflow as HumAL but in a self-contained Colab environment using the 20newsgroups dataset instead of real ticket data. The correspondence is:

HumAL Component	Hackathon Notebook Equivalent
Dispatch Labeling UI	`learner.query(X_pool, n_instances=N)` ,Äî selects most uncertain samples
Human operator labeling	`y_pool[query_idx]` ,Äî simulation using true labels
Training page (retrain)	`learner.teach(X_pool[query_idx], new_labels)`
Pool management	`X_pool = np.delete(X_pool, query_idx, axis=0)`
LIME explanations	`LimeTextExplainer.explain_instance(text, predict_fn)`
Ticket classifier	`LogisticRegression` trained on TF-IDF features
Uncertainty sampling	`entropy_sampling` (modAL)

6.1 Why entropy_sampling Aligns with HumAL

HumAL's dispatch labeling interface surfaces tickets using uncertainty-based selection. For a 4-class routing problem, entropy sampling is the most complete measure of uncertainty (considers all class probabilities), which is why it is the recommended query strategy in the hackathon.

6.2 Why LIME is Used in Both HumAL and the Notebook

HumAL uses LIME for local text explanations because it works directly with raw text ,Äî no preprocessing is required for the explanation algorithm itself. The `predict_fn` closure bridges the gap between raw text (what LIME perturbs) and TF-IDF features (what the model needs), exactly as implemented in Cell 5 of the notebook.

---

7 ,Äî Documentation References

- **Architecture details:** `docs/ARCHITECTURE.md`
- **REST API reference:** `docs/API.md`
- **User guide (step-by-step):** `docs/USER_GUIDE.md`
- **Development setup:** `docs/DEVELOPMENT.md`
- **Docker deployment:** `docs/DEPLOYMENT.md`

---

8 ,Äî Citation

```bibtex
@INPROCEEDINGS{11096208,
  author={Fatouros, George and Makridis, Georgios and Kousiouris, George and
          Soldatos, John and Tsadimas, Anargyros and Kyriazis, Dimosthenis},
  booktitle={2025 21st International Conference on Distributed Computing in
             Smart Systems and the Internet of Things (DCOSS-IoT)},
  title={Towards Conversational AI for Human-Machine Collaborative MLOps},
  year={2025},
  pages={1079-1086},
  doi={10.1109/DCOSS-IoT65416.2025.00162}
}
```

HumAIne EU Project ,Äî Grant Agreement 101120218
For RAG indexing into HumAIne Swarm chatbot